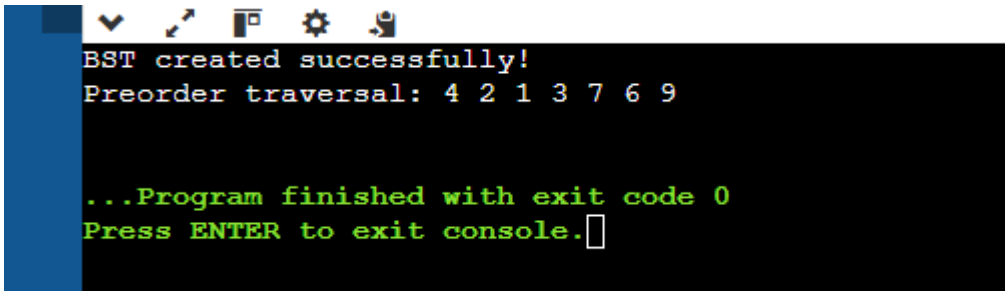
```
#include <stdio.h>
#include <stdlib.h>

struct TreeNode {
    int val;
    struct TreeNode *left;
    struct TreeNode *right;
};

struct TreeNode* createNode(int val) {
    struct TreeNode* node = malloc(sizeof(struct TreeNode));

    if (node == NULL) {
        exit(1);
    }

    node->val = val;
    node->left = NULL;
    node->right = NULL;

    return node;
}

struct TreeNode* insertIntoBST(struct TreeNode* root, int val) {
    if (root == NULL) {
        return createNode(val);
    }
```

```
    if (val < root->val) {
        root->left = insertIntoBST(root->left, val);
    }
    else if (val > root->val) {
        root->right = insertIntoBST(root->right, val);
    }

    return root;
}

void printTree(struct TreeNode* root) {
    if (root == NULL) return;

    printf("%d ", root->val);
    printTree(root->left);
    printTree(root->right);
}

void freeTree(struct TreeNode* root) {
    if (root == NULL) return;

    freeTree(root->left);
    freeTree(root->right);
    free(root);
}

int main() {
    int values[] = {4, 2, 7, 1, 3, 6, 9};
```

```
    int n = 7;

    struct TreeNode* root = NULL;

    for (int i = 0; i < n; i++) {
        root = insertIntoBST(root, values[i]);
    }

    printf("BST created successfully!\n");
    printf("Preorder traversal: ");
    printTree(root);

    printf("\n");

    freeTree(root);

    return 0;
}
```